

# WhoPenn: Project Report

Josh Ludan, Hanxi Chen, Yiwei Yan, Yuxuan Gao

## System Overview

“WhoPenn” is a social networking website that allows users to find out who’s who at Penn. WhoPenn allows users to create posts, chat with their friends, and browse the news. When a user first signs up to the system, they can use FriendFinder to send friend requests to users. While waiting for their friend to accept their request, they can post on their wall or browse and search for live news specifically tailored towards their interests. When their friend accepts their request, they will be able to see each other’s posts and the user can now post on their new friend’s wall. Additionally, the two users can use the chat features to talk to each other. As a user gains more friends, their network will expand and they will be able to see this in the visualizer. More friends also mean more interactions as they comment on each other’s posts and post/comment on each other’s walls. Users are also regularly notified of major events such as friend requests and chat invitation requests.

## Technical Description

WhoPenn uses the following major components: NodeJS & EJS for the front-end framework, DynamoDB to handle information storage, Spark & Livy to generate news recommendations for users, and Socket.io for the chats.

The system uses a total of 15 Dynamodb tables to maintain all system data. Below are descriptions of each table along with short descriptions of the decision considerations that explain that table’s form where necessary.

- User Tables:
  - users table: All user data is stored on a singular user table keyed on their userid. This table stores their information as well as their hashed password & associated salt, username, online status, and list of chat ids.
  - friends: This table is keyed on userid with friendid as a sort key. This table is a simple edge list that encodes friendship information.
- News Tables:
  - news: This table is keyed on newsid and stores information on articles. It also has a secondary index with a timestamp as a sort key to allow querying by category which is necessary when getting the newsfeed for a particular person with particular interests.
  - newslikes: This table is keyed on newsid with userid as a sort key. It allows us to tally which users liked a news article.
  - newsweights: This table is keyed on newsid with userid as a sort key. It stores the weight of that article for a given user. This is utilized by our recommendation system. This table is updated hourly via a spark job

- invertedNews: This table is keyed on keywords with newsid as a sort key. It's used by the news search functionality to identify which article headlines contain a given keyword.
- Chat Tables:
  - chats: This table, keyed on chatid, stores information about each chat such as its members, name, and latest message and messageid.
  - chat-messages: This table is keyed on messageid and stores the details of each message sent. It has chatid as a secondary key with timestamp as a sort key to allow for querying by chatid. This allows us to get the latest messages for a given chat.
- Posts:
  - posts: This table is keyed on postid. Each entry contains the content, timestamp, and creator for a given post.
  - user-posts: This table is keyed on userid with timestamp as a sort key. It contains the destination for a given userpost and the postid associated with the user post.
  - postlikes: This table is keyed on newsid with userid as a sort key. It has no other attributes and allows us to tally which users liked a post.
  - comments: This table has commentid as a primary key. It also has postid as a secondary key with timestamp as a sort key. This allows us to get the latest comments for a given postid. In addition to this, each entry stores the comment content and creator as an attribute.
- Notifications:
  - notifications: This table is keyed on notificationid. It stores a notification's content, status, type, and timestamp. This table has status-userid as a secondary key with timestamp as a sort key. It also has status-type-userid as a tertiary key with timestamp as a sort key. These other keys allow for more complex queries on the table such as seeing only unseen friend request notifications.
  - pendingFriendRequests: This table is keyed on notification receiverid as a primary key with senderid as a sort key. It has notificationid as an attribute. This table increases general security by preventing duplicate friend requests from being sent. Additionally, before each call to the friends table adds an edge they have to check with this table to ensure that a request was sent
  - pendingChatRequests: This table is keyed on notification receiverid as a primary key with chatid as a sort key. It has notificationid as an attribute. This table also improves security the same way the pendingFriendsRequests table does.

## News Recommendation Adsorption Algorithm

The adsorption algorithm leverages information on article likes, user interests, and the friendship graph to rank the degree of relevance of each article for recommendations. We initiate a uniquely labeled weight of 1.0 for each user. Then, we use bulk operations supported by Spark RDD to compute the distribution of weights throughout iterations. The termination condition has two folds - either the number of maximum iterations (currently set as 15) is achieved, or convergence in the weight matrix appears. For convergence, we use an outer join to compute the difference in each node's all types of weight between the current and the previous round. This outer join ensures that we also cover all new types of weights gained in the new

round. For edge cases of users with no friends or no article likes, our algorithm focuses on the interest categories selected and generates recommendations from a uniform distribution within each category. To run adsorption periodically, we decided to run a while loop to initiate a Spark job every 60 minutes.

## Chat Socket Functionality

This chat room uses a combination of socket and dynamoDB. It will record the message and chat the user creates and every other activity (kicking user, leaving chat, invite user, delete chat, etc.) on dynamoDB. After the confirmation from db, it would emit through the socket. We use socket.io, where we create rooms based on chatid, and emit specific messages (“join room”, “chat message”, “kick user”, etc.) with specific content. The chat functions in a “lazy” manner, i.e. once a user’s chat receive a message from socket.io, it will respond in some pre-described way (receiving “send message” will make the chat to update the message on div.mid-app and div.left-app, receiving “delete room” will delete the room and reload the chat sessions, etc.)

## Design Decisions

Lots of considerations were taken to ensure the security, scalability, and robustness of the system. These considerations are outlined below:

### Security & Robustness

- Password hashing
  - The passwords are salted and hashed with SHA-256 to ensure that the passwords remain safe and secure.
- Generating random IDS
  - The ids within the table are generated from hashes incorporating the current time and the item content string. This generates a pseudorandom id that allows items with the same properties to be stored on the same dynamoDB table.
- Securing routes
  - Most routes only work if a user is currently logged in with a cookie. Users who are not in a session are always redirected to the login page. The same is true for our post and get routes.
- User privilege checks
  - In addition to checking if a user has a cookie, routes that access potentially sensitive information check the details of the session to see if a user should be allowed to access given information (for example, you can only a user page of you are that user or their friend)
- Returning only relevant details
  - The details returned by each route match the privilege of a user. For example, if you are not a friend of a given user you can only access their name and affiliation. If you are their friend, you can access things such as their birthday and interests.
- Resilience to spoofed requests
  - We designed two inverted tables (pendingChatRequests and pendingFriendRequests) that allow us to efficiently search if a chat invite request or a friend request has actually been sent before allowing a user to send a join request. A user who wishes to send an invite

will log that request on these tables and if a join request is sent that doesn't correspond to an entry on this table, the request will be denied.

## Scalability

- DynamoDB & EC2
  - These technologies allow us to naturally scale our capabilities to the demand. Currently, some of our tables are provisioned and some are scaled. In the case of a demand spike, we will get notifications that will inform us when we need to provision more for a given table.
- Minimizing the number of computations
  - We designed our tables carefully to ensure that we can obtain the information that we need in the least number of db calls possible. In any worst case, the number of computations for most regular operations such as pulling posts will scale linearly in proportion to the demand. We do this by carefully considering the use cases for each table and designing secondary indices to cover all use cases.

## Design Alterations

### Atomizing Features

- We initially planned on making the user experience much more cohesive by having most of the functionality on as few pages as possible. For example, we wanted the user homepage, wall, and news to all be on the same page and have users be able to switch between them by clicking tabs.
- This made splitting up the work for our group much more difficult. We found that work became easier to parallelize if we let each major functionality exist on its own separate page. This explains why the search page and friends page for example are separate from the main page.

### Periodic Adsorption Computation

- We initially planned to use AWS EMR's interface to schedule periodic computation jobs on Spark, but we realized that this would add overhead to configuration and dependencies when we migrate our service to EC2. Therefore, we decided to instead have a Java file continuously running to submit a Spark job every hour. This reduces compatibility concerns and does not cause scalability issues because the actual computation is still conducted in the cloud through Livy.

# Login Page

Login

Enter username here:  
Michael

Enter password here:  
\*\*\*\*\*

Login

[Click here to sign up](#)

A screenshot of a Facebook profile page for a user named 'pop'. The header shows navigation links: Home, Friends, Groups, Pages, Marketplace, and More. The profile picture is a blue circle with the letter 'p'. The cover photo is a solid blue rectangle. The name 'pop' is displayed in large bold letters, with the username '@pop' below it. The bio reads 'We're excited'. There is a 'Comment' button and a 'Like' button with a count of 1. A post is visible with the text 'I'm' and a timestamp of '12/20/2021 9:45 AM'. The post has a 'Comment' button and a 'Like' button with a count of 1.

**Sign Up**

Email

Password

Confirm Password

First Name  Last Name

Login Name  Affiliation

**I agree to the Terms & Conditions**

☒ I have entered an invalid email address!

---

Birthday

Choose your passions:

|               |                |               |               |               |         |
|---------------|----------------|---------------|---------------|---------------|---------|
| Entertainment | Politics       | World News    | Sports        | Business      | Science |
| Tech          | Style & Beauty | Style         | Comedy        | Games         | Impact  |
| Arts          | Acts & Culture | Media         | Queer Voices  | Latino Voices |         |
| Black Voices  | Wellness       | Mental Health | Parenting     | Taste         |         |
| Food & Drink  | Workplace      | Travel        | Home & Living | Career        |         |
| Worldviews    | Diverse        | Parents       | Crime         | World News    | Fifty   |
| College       | Education      | Religion      |               |               |         |

The screenshot displays a web application interface. At the top, there is a navigation bar with links for Search Friends, Search News, and Location. Below this, the main content area is divided into two columns. The left column features a 'Welcome, pop' message, a 'Post something' section with a text input field and a 'Submit' button, and a 'Posts' section showing a post by 'pop' from 12/20/2021 at 9:37 AM with a 'Comment' button. The right column is titled 'Notifications' and lists three notifications: one from 'bob' about a new friend, one from 'trent' about a new post, and another from 'bob' about a new post. Each notification includes a timestamp and buttons for 'Accept' and 'Reject'.

## Profile Update

New Email

Old Password

Affiliation

Choose your new interests

☐ Entertainment
 ☐ Politics
 ☐ Tech
 ☐ Arts & Culture
 ☐ Wellness
 ☐ Travel
 ☐ Crime

☐ Style & Beauty
 ☐ Comedy
 ☐ Media
 ☐ Healthy Living
 ☐ Home & Living
 ☐ Weird News

☐ Sports
 ☐ Women
 ☐ Dating
 ☐ Parenting
 ☐ Food & Drink
 ☐ Fitness

☐ Business
 ☐ Impact
 ☐ Latino Voices
 ☐ Turn of Mind
 ☐ Weddings
 ☐ Filly

☐ Science
 ☐ Black Voices
 ☐ Workpod
 ☐ Disrace
 ☐ Education
 ☐ Religion

[Change Email](#)

[Change Password](#)

[Change Affiliation](#)

[Change Interests](#)

Search

josh josh

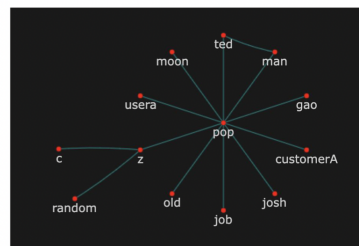
job job


[Search Friends](#)
[Search News](#)
[Visualizer](#)

---

**Friend List (10)**
[Edit Friends](#)

|                                                                                                                                                  |                                                                                                                                                               |
|--------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <p><b>man</b><br/>Name: man man<br/>Affiliation: Wharton</p> |  <p><b>customerw</b><br/>Name: customer A<br/>Affiliation: Engineering</p> |
|  <p><b>add</b><br/>Name: old old<br/>Affiliation: Penn</p>    |  <p><b>josh</b><br/>Name: josh josh<br/>Affiliation: Penn</p>              |
|  <p><b>gap</b><br/>Name: ga gap<br/>Affiliation: Wharton</p>  |  <p><b>users</b><br/>Name: users users<br/>Affiliation: College</p>        |
|  <p><b>ted</b><br/>Name: ted ted<br/>Affiliation: Law</p>     |  <p><b>moon</b><br/>Name: moon moon<br/>Affiliation: Medicine</p>          |
|  <p><b>job</b><br/>Name: job job<br/>Affiliation: Penn</p>    |  <p><b>first</b><br/>Name: first last<br/>Affiliation: Penn</p>            |



## News Recommendation

## Our News

16/12/2021 2:00 PM ENTERTAINMENT

**Hayley Miller**

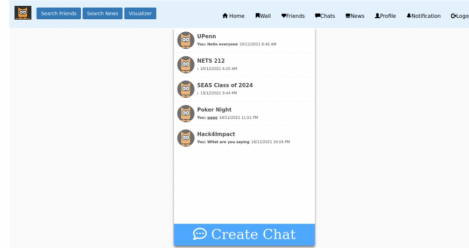
**Gene Simmons Denies Sexual Misconduct Allegations**

Likes 1 

Likes 2 

Leigh Bickley  
'The Voice' Is Great TV. It's Just Not Always Great For The Winners.  
Likes 2 

Chat Cover Page



## News Search

## News Search

atom Q

Number of results for "atom" : 13

15/2/2018 2:00 PM WORLD NEWS

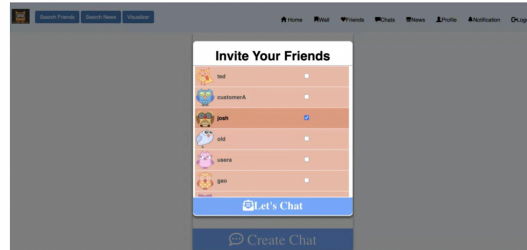
**Carol Kuruvilla**  
[Oxford Student Wins Prize For Photo Of Atom Taken With DSLR Camera](#)

Likes 0 

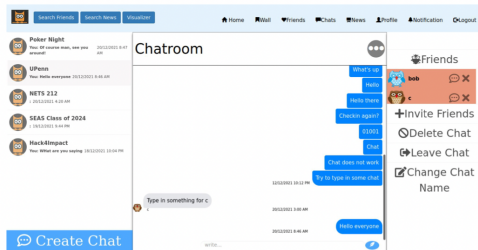
30/7/2017 3:00 PM ENTERTAINMENT

Likes 3 

## Chat Invitation



## Group Chat Page



## Change Chat Name

